```
package com.chenpp.graph.admin.controller;

import com.chenpp.graph.admin.model.ImportResult;
import com.chenpp.graph.admin.model.Result;
import com.chenpp.graph.admin.model.GraphVertexDef;
import com.chenpp.graph.admin.model.GraphEdgeDef;
import com.chenpp.graph.admin.model.PageResult;
import com.chenpp.graph.admin.service.GraphDataService;
import com.chenpp.graph.admin.service.GraphSchemaService;
import com.chenpp.graph.core.model.GraphEdge;
import com.chenpp.graph.core.model.GraphSummary;
import com.chenpp.graph.core.model.GraphVertex;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;
import java.util.Map;

/**
 * 图数据管理 API
 */
@SuppressWarnings("unchecked")
@RestController
@RequestMapping("/api/graph/data")
public class GraphDataController {

    @Autowired
    private GraphDataService graphDataService;

    @Autowired
    private GraphSchemaService graphSchemaService;

    @PostMapping("/importVertices")
    public Result<ImportResult> importVertexData(
            @RequestParam(required = false) Long graphId,
            @RequestParam(required = false) Long vertexTypeId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestParam(required = false) String label,
            @RequestPart("file") MultipartFile file) {
        if (file == null || file.isEmpty()) {
            return Result.error("CSV 文件不能为空");
        }
        if (!((connectionId != null && graphCode != null && label != null) || (graphId != null && vertexTypeId != null))) {
            return Result.error("需要提供 (graphId + vertexTypeId) 或 (connectionId + graphCode + label)");
        }
        ImportResult result = graphDataService.importVertexData(graphId, vertexTypeId, connectionId, graphCode, label,
file);
        return Result.success(result);
    }

    @PostMapping("/importEdges")
    public Result<ImportResult> importEdgeData(
            @RequestParam(required = false) Long graphId,
            @RequestParam(required = false) Long edgeTypeId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestParam(required = false) String label,
            @RequestPart("file") MultipartFile file) {
```

```
        if (file == null || file.isEmpty()) {
            return Result.error("CSV 文件不能为空");
        }
        if (!((connectionId != null && graphCode != null && label != null) || (graphId != null && edgeTypeId != null))) {
            return Result.error("需要提供 (graphId + edgeTypeId) 或 (connectionId + graphCode + label)");
        }
        ImportResult result = graphDataService.importEdgeData(graphId, edgeTypeId, connectionId, graphCode, label,
file);
        return Result.success(result);
    }

    @GetMapping("/vertices")
    public Result<PageResult<GraphVertex>> getVertexDataList(
            @RequestParam Long graphId,
            @RequestParam Long vertexTypeId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestParam(defaultValue = "1") Integer page,
            @RequestParam(defaultValue = "10") Integer size) {
        String label = null;
        if (vertexTypeId != null && vertexTypeId < 0) {
            if (connectionId != null && graphCode != null) {
                List<GraphVertexDef> vertexDefs = graphSchemaService.discoverVertexDefs(graphId, connectionId,
graphCode);
                for (GraphVertexDef vd : vertexDefs) {
                    if (vd.getId().equals(vertexTypeId)) {
                        label = vd.getLabel();
                        break;
                    }
                }
            }
            if (label == null) {
                return Result.success(PageResult.empty(page, size));
            }
        }

        PageResult<GraphVertex> data = graphDataService.queryVertexDataList(graphId, vertexTypeId, label, page, size,
connectionId, graphCode);
        return Result.success(data);
    }

    @GetMapping("/edges")
    public Result<PageResult<GraphEdge>> getEdgeDataList(
            @RequestParam Long graphId,
            @RequestParam Long edgeTypeId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestParam(defaultValue = "1") Integer page,
            @RequestParam(defaultValue = "10") Integer size) {
        String label = null;
        if (edgeTypeId != null && edgeTypeId < 0) {
            if (connectionId != null && graphCode != null) {
                List<GraphEdgeDef> edgeDefs = graphSchemaService.discoverEdgeDefs(graphId, connectionId,
graphCode);
                for (GraphEdgeDef ed : edgeDefs) {
                    if (ed.getId().equals(edgeTypeId)) {
                        label = ed.getLabel();
                        break;
                    }
                }
```

```
            }
            if (label == null) {
                return Result.success(PageResult.empty(page, size));
            }
        }
        PageResult<GraphEdge> data = graphDataService.queryEdgeDataList(graphId, edgeTypeId, label, page, size,
connectionId, graphCode);
        return Result.success(data);
    }

    @PostMapping("/vertex")
    public Result<Boolean> addVertexData(
            @RequestParam Long graphId,
            @RequestParam Long vertexTypeId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestBody Map<String, Object> data) {
        boolean result = graphDataService.addVertexData(graphId, vertexTypeId, connectionId, graphCode, data);
        if (!result) {
            return Result.error("新增顶点数据失败");
        }
        return Result.success(true);
    }

    @PostMapping("/edge")
    public Result<Boolean> addEdgeData(
            @RequestParam Long graphId,
            @RequestParam Long edgeTypeId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestBody Map<String, Object> data) {
        boolean result = graphDataService.addEdgeData(graphId, edgeTypeId, connectionId, graphCode, data);
        if (!result) {
            return Result.error("新增边数据失败");
        }
        return Result.success(true);
    }

    @PutMapping("/vertex")
    public Result<Boolean> updateVertexData(
            @RequestParam Long graphId,
            @RequestParam String vertexId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestBody Map<String, Object> data) {
        boolean result = graphDataService.updateVertexData(graphId, vertexId, connectionId, graphCode, data);
        if (!result) {
            return Result.error("更新顶点数据失败");
        }
        return Result.success(true);
    }

    @PutMapping("/edge")
    public Result<Boolean> updateEdgeData(
            @RequestParam Long graphId,
            @RequestParam String edgeId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode,
            @RequestBody Map<String, Object> data) {
        boolean result = graphDataService.updateEdgeData(graphId, edgeId, connectionId, graphCode, data);
```

```
        if (!result) {
            return Result.error("更新边数据失败");
        }
        return Result.success(true);
    }

    @DeleteMapping("/vertex")
    public Result<Boolean> deleteVertex(
            @RequestParam Long graphId,
            @RequestParam String vertexId,
            @RequestParam(required = false) String label,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode) {
        boolean result = graphDataService.deleteVertex(graphId, vertexId, label, connectionId, graphCode);
        if (!result) {
            return Result.error("删除顶点失败");
        }
        return Result.success(true);
    }

    @DeleteMapping("/edge")
    public Result<Boolean> deleteEdge(
            @RequestParam Long graphId,
            @RequestParam String edgeId,
            @RequestParam(required = false) String label,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode) {
        boolean result = graphDataService.deleteEdge(graphId, edgeId, label, connectionId, graphCode);
        if (!result) {
            return Result.error("删除边失败");
        }
        return Result.success(true);
    }

    @GetMapping("/summary")
    public Result<GraphSummary> getGraphSummary(
            @RequestParam Long graphId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode) {
        GraphSummary summary = graphDataService.getGraphSummary(graphId, connectionId, graphCode);
        return Result.success(summary);
    }
}

package com.chenpp.graph.admin.service.impl;

import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.chenpp.graph.admin.enums.GraphTypeEnum;
import com.chenpp.graph.admin.model.GraphInfo;
import com.chenpp.graph.admin.model.GraphConnection;
import com.chenpp.graph.admin.model.GraphEdgeDef;
import com.chenpp.graph.admin.model.GraphVertexDef;
import com.chenpp.graph.admin.model.GraphPropertyDef;
import com.chenpp.graph.admin.model.ImportResult;
import com.chenpp.graph.admin.model.PageResult;
import com.chenpp.graph.admin.service.GraphDataService;
import com.chenpp.graph.admin.service.GraphConnectionService;
import com.chenpp.graph.admin.service.GraphEdgeDefService;
import com.chenpp.graph.admin.service.GraphVertexDefService;
import com.chenpp.graph.admin.service.GraphPropertyDefService;
```

```
import com.chenpp.graph.admin.service.GraphService;
import com.chenpp.graph.admin.util.GraphClientFactory;
import com.chenpp.graph.core.GraphClient;
import com.chenpp.graph.core.GraphDataOperations;
import com.chenpp.graph.core.constant.GraphConstants;
import com.chenpp.graph.core.model.GraphConf;
import com.chenpp.graph.core.model.GraphData;
import com.chenpp.graph.core.model.GraphEdge;
import com.chenpp.graph.core.model.GraphSummary;
import com.chenpp.graph.core.model.GraphVertex;
import com.chenpp.graph.core.util.DataTypeConverter;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import org.apache.commons.csv.CSVFormat;
import org.apache.commons.csv.CSVParser;
import org.apache.commons.csv.CSVRecord;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;


@Slf4j
@Service
public class GraphDataServiceImpl implements GraphDataService {

    @Autowired
    private GraphService graphService;

    @Autowired
    private GraphConnectionService connectionService;

    @Autowired
    private GraphVertexDefService vertexDefService;

    @Autowired
    private GraphEdgeDefService edgeDefService;

    @Autowired
    private GraphPropertyDefService propertyDefService;

    private final Map<String, ReentrantLock> importLocks = new ConcurrentHashMap<>();

    private static final int BATCH_SIZE = 100;

    private ReentrantLock getImportLock(Long graphId, String graphCode) {
        String key = graphId != null ? "graphId:" + graphId : "graphCode:" + graphCode;
```

```
        return importLocks.computeIfAbsent(key, k -> new ReentrantLock());
    }

    private void releaseImportLock(Long graphId, String graphCode) {
        String key = graphId != null ? "graphId:" + graphId : "graphCode:" + graphCode;
        importLocks.remove(key);
    }

    @Override
    public ImportResult importVertexData(Long graphId, Long vertexTypeId, Long connectionId, String graphCode, String
label, MultipartFile file) {
        return doImportData(graphId, vertexTypeId, connectionId, graphCode, label, file, true);
    }

    @Override
    public ImportResult importEdgeData(Long graphId, Long edgeTypeId, Long connectionId, String graphCode, String
label, MultipartFile file) {
        return doImportData(graphId, edgeTypeId, connectionId, graphCode, label, file, false);
    }

    private ImportResult doImportData(Long graphId, Long typeId, Long connectionId, String graphCode,
                                      String label, MultipartFile file, boolean isVertex) {
        String dataType = isVertex ? "顶点" : "边";
        ImportResult result = new ImportResult();
        result.setTotalCount(0);
        result.setSuccessCount(0);
        result.setFailureCount(0);
        List<String> errorMessages = new ArrayList<>();

        ReentrantLock importLock = getImportLock(graphId, graphCode);
        boolean lockAcquired = false;
        try {
            if (!importLock.tryLock(30, TimeUnit.SECONDS)) {
                errorMessages.add("系统繁忙，请稍后再试");
                result.setErrorMessages(errorMessages.toArray(new String[0]));
                return result;
            }
            lockAcquired = true;
            log.info("获取导入锁成功，graphId={}, graphCode={}", graphId, graphCode);

            ImportContext context = prepareImportContext(graphId, typeId, connectionId, graphCode, label, isVertex);
            if (context.hasError()) {
                result.setErrorMessages(context.getErrors().toArray(new String[0]));
                return result;
            }

            List<Map<String, String>> dataList = parseDataCsv(file);
            result.setTotalCount(dataList.size());

            GraphClient graphClient = GraphClientFactory.createGraphClient(context.getGraphConf());
            GraphDataOperations graphDataOperations = graphClient.opsForGraphData();

            int successCount = isVertex
                    ? batchImportVertices(graphDataOperations, dataList, context.getLabel(), errorMessages)
                    : batchImportEdges(graphDataOperations, dataList, context.getLabel(), errorMessages);

            result.setSuccessCount(successCount);
            result.setFailureCount(dataList.size() - successCount);
            result.setErrorMessages(errorMessages.toArray(new String[0]));
```

```
                log.info("导入{}数据完成，总数={}，成功={}，失败={}", dataType, dataList.size(), successCount, dataList.size() -
successCount);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                log.error("获取导入锁被中断", e);
                errorMessages.add("导入操作被中断");
                result.setErrorMessages(errorMessages.toArray(new String[0]));
            } catch (Exception e) {
                log.error("导入{}数据失败", dataType, e);
                errorMessages.add("导入" + dataType + "数据失败: " + e.getMessage());
                result.setErrorMessages(errorMessages.toArray(new String[0]));
            } finally {
                if (lockAcquired) {
                    importLock.unlock();
                    releaseImportLock(graphId, graphCode);
                    log.info("释放导入锁，graphId={}, graphCode={}", graphId, graphCode);
                }
            }

            return result;
        }

        /**
         * 批量导入顶点数据
         */
        private int batchImportVertices(GraphDataOperations graphDataOperations, List<Map<String, String>> dataList, String
label, List<String> errorMessages) {
            int successCount = 0;
            List<GraphVertex> batch = new ArrayList<>(BATCH_SIZE);

            for (int i = 0; i < dataList.size(); i++) {
                Map<String, String> dataRow = dataList.get(i);
                try {
                    GraphVertex vertex = new GraphVertex();
                    vertex.setUid(dataRow.get(GraphConstants.UID));
                    vertex.setLabel(label);
                    vertex.setProperties(extractProperties(new HashMap<>(dataRow), Set.of(GraphConstants.LABEL)));
                    batch.add(vertex);

                    if (batch.size() >= BATCH_SIZE || i == dataList.size() - 1) {
                        try {
                            if (batch.size() > 1) {
                                graphDataOperations.addVertices(batch);
                                successCount += batch.size();
                            } else {
                                graphDataOperations.addVertex(batch.get(0));
                                successCount++;
                            }
                        } catch (Exception batchEx) {
                            log.warn("批量导入失败，尝试逐条插入: {}", batchEx.getMessage());
                            for (int j = 0; j < batch.size(); j++) {
                                GraphVertex v = batch.get(j);
                                try {
                                    graphDataOperations.addVertex(v);
                                    successCount++;
                                } catch (Exception singleEx) {
                                    int rowNum = (i - batch.size() + j + 2);
                                    log.error("导入顶点数据失败，第{}行: {}", rowNum, singleEx.getMessage(), singleEx);
                                    errorMessages.add(String.format("第%d行导入失败：%s", rowNum,
singleEx.getMessage()));
```

```
                            }
                        }
                    }
                    batch.clear();
                }
            } catch (Exception e) {
                log.error("导入顶点数据失败，第{}行: {}", i + 2, e.getMessage(), e);
                errorMessages.add(String.format("第%d 行导入失败: %s", i + 2, e.getMessage()));
            }
        }

        return successCount;
    }


    private ImportContext prepareImportContext(Long graphId, Long typeId, Long connectionId, String graphCode, String label, boolean isVertex) {
        ImportContext context;

        if (connectionId != null && graphCode != null && label != null) {
            GraphConnection connection = connectionService.getById(connectionId);
            if (connection == null) {
                context = new ImportContext(null, null, null);
                context.addError("图数据库连接不存在，connectionId=" + connectionId);
                return context;
            }
            context = new ImportContext(connection, graphCode, label);
        } else if (graphId != null && typeId != null) {
            GraphInfo graphInfo = graphService.getById(graphId);
            if (graphInfo == null) {
                context = new ImportContext(null, null, null);
                context.addError("图不存在，graphId=" + graphId);
                return context;
            }

            GraphConnection connection = connectionService.getById(graphInfo.getConnectionId());
            if (connection == null) {
                context = new ImportContext(null, null, null);
                context.addError("图数据库连接不存在，connectionId=" + graphInfo.getConnectionId());
                return context;
            }

            String targetLabel;
            if (isVertex) {
                GraphVertexDef vertexDef = vertexDefService.getById(typeId);
                if (vertexDef == null) {
                    context = new ImportContext(null, null, null);
                    context.addError("顶点类型不存在，vertexTypeId=" + typeId);
                    return context;
                }
                targetLabel = vertexDef.getLabel();
            } else {
                GraphEdgeDef edgeDef = edgeDefService.getById(typeId);
                if (edgeDef == null) {
                    context = new ImportContext(null, null, null);
                    context.addError("边类型不存在，edgeTypeId=" + typeId);
                    return context;
                }
                targetLabel = edgeDef.getLabel();
            }
```

```
            context = new ImportContext(connection, graphInfo.getCode(), targetLabel);
        } else {
            context = new ImportContext(null, null, null);
            context.addError("缺少必要参数，需要提供 (graphId + typeId) 或 (connectionId + graphCode + label)");
        }

        return context;
    }


    /**
     * 检测字符串中是否包含中文乱码（出现非法 UTF-8 替换字符 U+FFFD）
     */
    private boolean containsGarbledChinese(String text) {
        if (text == null || text.isEmpty()) {
            return false;
        }
        if (text.contains("\uFFFD")) {
            return true;
        }
        int strangeCount = 0;
        for (char c : text.toCharArray()) {
            if (c == '\uFFFD' || (c > 0x7F && c < 0xA0)) {
                strangeCount++;
            }
        }
        return strangeCount > 3;
    }

    private List<Map<String, String>> parseDataCsv(MultipartFile file) throws Exception {
        List<Map<String, String>> dataList = new ArrayList<>();

        byte[] rawBytes = file.getBytes();
        String content = new String(rawBytes, StandardCharsets.UTF_8);
        Charset charset = StandardCharsets.UTF_8;
        if (containsGarbledChinese(content)) {
            charset = Charset.forName("GBK");
        }

        try (BufferedReader reader = new BufferedReader(new InputStreamReader(file.getInputStream(), charset));
             CSVParser csvParser = new CSVParser(reader, CSVFormat.DEFAULT.builder()
                     .setHeader()
                     .setSkipHeaderRecord(true)
                     .setIgnoreHeaderCase(true)
                     .setTrim(true)
                     .build())) {

            List<String> headers = csvParser.getHeaderNames();

            for (CSVRecord record : csvParser) {
                Map<String, String> dataMap = new HashMap<>();
                for (String header : headers) {
                    String value = record.get(header);
                    dataMap.put(header, value != null ? value.trim() : null);
                }
                dataList.add(dataMap);
            }
        }
```

```
        return dataList;
    }


    private int batchImportEdges(GraphDataOperations graphDataOperations, List<Map<String, String>> dataList, String label, List<String> errorMessages) {
        int successCount = 0;
        List<GraphEdge> batch = new ArrayList<>(BATCH_SIZE);

        for (int i = 0; i < dataList.size(); i++) {
            Map<String, String> dataRow = dataList.get(i);
            try {
                GraphEdge edge = new GraphEdge();
                edge.setUid(dataRow.get(GraphConstants.UID));
                edge.setLabel(label);
                edge.setStartUid(dataRow.get(GraphConstants.START_UID));
                edge.setEndUid(dataRow.get(GraphConstants.END_UID));
                edge.setProperties(extractProperties(dataRow, Set.of(GraphConstants.START_UID, GraphConstants.END_UID, GraphConstants.LABEL)));
                batch.add(edge);

                if (batch.size() >= BATCH_SIZE || i == dataList.size() - 1) {
                    try {
                        if (batch.size() > 1) {
                            graphDataOperations.addEdges(batch);
                            successCount += batch.size();
                        } else {
                            graphDataOperations.addEdge(batch.get(0));
                            successCount++;
                        }
                    } catch (Exception batchEx) {
                        log.warn("批量导入失败，尝试逐条插入: {}", batchEx.getMessage());
                        for (int j = 0; j < batch.size(); j++) {
                            GraphEdge e = batch.get(j);
                            try {
                                graphDataOperations.addEdge(e);
                                successCount++;
                            } catch (Exception singleEx) {
                                int rowNum = (i - batch.size() + j + 2);
                                log.error("导入边数据失败，第{}行: {}", rowNum, singleEx.getMessage(), singleEx);
                                errorMessages.add(String.format("第%d行导入失败: %s", rowNum, singleEx.getMessage()));
                            }
                        }
                    }
                    batch.clear();
                }
            } catch (Exception e) {
                log.error("导入边数据失败，第{}行: {}", i + 2, e.getMessage(), e);
                errorMessages.add(String.format("第%d 行导入失败: %s", i + 2, e.getMessage()));
            }
        }

        return successCount;
    }


    public PageResult<GraphVertex> queryVertexDataList(Long graphId, Long vertexTypeId, String label, Integer page, Integer size, Long connectionId, String graphCode) {
        try {
```

```
            if (label == null) {
                GraphVertexDef vertexDef = vertexDefService.getById(vertexTypeId);
                if (vertexDef == null) {
                    log.error("顶点类型不存在，vertexTypeId={}", vertexTypeId);
                    return PageResult.empty(page, size);
                }
                label = vertexDef.getLabel();
            }

            GraphDataOperations ops = getGraphDataOperations(graphId, connectionId, graphCode);
            int skip = (page - 1) * size;

            long total = ops.countVertices(label);

            String query = buildLabelQuery(graphId, connectionId, label, skip, size);
            GraphData graphData = ops.query(query);

            List<GraphVertex> records = (graphData == null || graphData.getVertices() == null)
                    ? new ArrayList<>()
                    : graphData.getVertices();

            return new PageResult<>(records, total, page, size);
        } catch (Exception e) {
            log.error("查询顶点数据列表失败，graphId={}, vertexTypeId={}", graphId, vertexTypeId, e);
            return PageResult.empty(page, size);
        }
    }

    public PageResult<GraphEdge> queryEdgeDataList(Long graphId, Long edgeTypeId, String label, Integer page,
Integer size, Long connectionId, String graphCode) {
        try {
            if (label == null) {
                GraphEdgeDef edgeDef = edgeDefService.getById(edgeTypeId);
                if (edgeDef == null) {
                    log.error("边类型不存在，edgeTypeId={}", edgeTypeId);
                    return PageResult.empty(page, size);
                }
                label = edgeDef.getLabel();
            }

            GraphDataOperations ops = getGraphDataOperations(graphId, connectionId, graphCode);
            int skip = (page - 1) * size;

            long total = ops.countEdges(label);

            String query = buildEdgeLabelQuery(graphId, connectionId, label, skip, size);
            GraphData graphData = ops.query(query);
            List<GraphEdge> records = (graphData == null || graphData.getEdges() == null)
                    ? new ArrayList<>()
                    : graphData.getEdges();
            return new PageResult<>(records, total, page, size);
        } catch (Exception e) {
            log.error("查询边数据列表失败，graphId={}, edgeTypeId={}", graphId, edgeTypeId, e);
            return PageResult.empty(page, size);
        }
    }

    @Override
    public boolean addVertexData(Long graphId, Long vertexTypeId, Long connectionId, String graphCode, Map<String,
Object> data) {
```

```
        try {
            GraphVertexDef vertexDef = vertexDefService.getById(vertexTypeId);
            if (vertexDef == null) {
                log.error("顶点类型不存在，vertexTypeId={}", vertexTypeId);
                return false;
            }
            GraphDataOperations ops = getGraphDataOperations(graphId, connectionId, graphCode);
            GraphVertex vertex = new GraphVertex();
            vertex.setLabel(vertexDef.getLabel());
            Map<String, Object> properties = handleProperties(data, vertexDef.getProperties());
            String uid = data.get(GraphConstants.UID).toString();
            vertex.setProperties(properties);
            vertex.setUid(uid);
            ops.addVertex(vertex);
            return true;
        } catch (Exception e) {
            log.error("新增顶点数据失败，graphId={}, vertexTypeId={}", graphId, vertexTypeId, e);
            return false;
        }
    }

    @Override
    public boolean addEdgeData(Long graphId, Long edgeTypeId, Long connectionId, String graphCode, Map<String,
Object> data) {
        try {
            GraphEdgeDef edgeDef = edgeDefService.getById(edgeTypeId);
            if (edgeDef == null) {
                log.error("边类型不存在，edgeTypeId={}", edgeTypeId);
                return false;
            }
            GraphDataOperations ops = getGraphDataOperations(graphId, connectionId, graphCode);

            Map<String, Object> properties = handleProperties(data, edgeDef.getProperties());
            String uid = data.get(GraphConstants.UID).toString();
            GraphEdge edge = new GraphEdge();
            edge.setLabel(edgeDef.getLabel());
            edge.setUid(uid);
            edge.setStartLabel(edgeDef.getStartLabel());
            edge.setEndLabel(edgeDef.getEndLabel());
            edge.setProperties(properties);
            ops.addEdge(edge);
            return true;
        } catch (Exception e) {
            log.error("新增边数据失败，graphId={}, edgeTypeId={}", graphId, edgeTypeId, e);
            return false;
        }
    }

    @Override
    public boolean updateVertexData(Long graphId, String vertexId, Long connectionId, String graphCode, Map<String,
Object> data) {
        try {
            GraphDataOperations ops = getGraphDataOperations(graphId, connectionId, graphCode);

            GraphVertex vertex = new GraphVertex();
            vertex.setUid(vertexId);
            if (data.containsKey(GraphConstants.LABEL)) {
                vertex.setLabel(data.get(GraphConstants.LABEL).toString());
            }
```

```
            String label = vertex.getLabel();
            List<GraphPropertyDef> propDefs = new ArrayList<>();
            if (label != null) {
                GraphVertexDef vertexDef = vertexDefService.getOne(
                        new QueryWrapper<GraphVertexDef>().eq("graph_id", graphId).eq("label", label)
                );
                if (vertexDef != null && vertexDef.getProperties() != null) {
                    propDefs = vertexDef.getProperties();
                }
            }

            Map<String, Object> properties = handleProperties(data, propDefs);
            vertex.setProperties(properties);
            ops.updateVertex(vertex);
            log.info("更新顶点成功，vertexId={}", vertexId);
            return true;
        } catch (Exception e) {
            log.error("更新顶点数据失败，graphId={}, vertexId={}", graphId, vertexId, e);
            return false;
        }
    }

    @Override
    public boolean updateEdgeData(Long graphId, String edgeId, Long connectionId, String graphCode, Map<String,
Object> data) {
        try {
            GraphDataOperations ops = getGraphDataOperations(graphId, connectionId, graphCode);

            GraphEdge edge = new GraphEdge();
            edge.setUid(edgeId);
            if (data.containsKey(GraphConstants.LABEL)) {
                edge.setLabel(data.get(GraphConstants.LABEL).toString());
            }

            String label = edge.getLabel();
            List<GraphPropertyDef> propDefs = new ArrayList<>();
            if (label != null) {
                GraphEdgeDef edgeDef = edgeDefService.getOne(
                        new QueryWrapper<GraphEdgeDef>().eq("graph_id", graphId).eq("label", label)
                );
                if (edgeDef != null && edgeDef.getProperties() != null) {
                    propDefs = edgeDef.getProperties();
                }
            }

            Map<String, Object> properties = handleProperties(data, propDefs);
            if (data.containsKey(GraphConstants.START_UID)) {
                edge.setStartUid(data.get(GraphConstants.START_UID).toString());
            } else if (properties.containsKey(GraphConstants.START_UID)) {
                edge.setStartUid(properties.get(GraphConstants.START_UID).toString());
            }
            if (data.containsKey(GraphConstants.END_UID)) {
                edge.setEndUid(data.get(GraphConstants.END_UID).toString());
            } else if (properties.containsKey(GraphConstants.END_UID)) {
                edge.setEndUid(properties.get(GraphConstants.END_UID).toString());
            }
            for (GraphPropertyDef propDef : propDefs) {
                String code = propDef.getCode();
                if (properties.containsKey(code)) {
                    properties.put(code, DataTypeConverter.convertValue(properties.get(code), propDef.getType()));
```

```
                }
            }
            edge.setProperties(properties);
            ops.updateEdge(edge);
            log.info("更新边成功，edgeId={}", edgeId);
            return true;
        } catch (Exception e) {
            log.error("更新边数据失败，graphId={}, edgeId={}", graphId, edgeId, e);
            return false;
        }
    }

    @Override
    public boolean deleteVertex(Long graphId, String vertexId, String label, Long connectionId, String graphCode) {
        if (graphId == null || vertexId == null) {
            return false;
        }
        try {
            GraphConf graphConf = GraphClientFactory.resolveGraphConf(graphId, connectionId, graphCode,
graphService, connectionService);
            GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
            GraphDataOperations graphDataOperations = graphClient.opsForGraphData();
            String vertexLabel = label;
            if (graphId > 0) {
                GraphVertexDef vertexDef = vertexDefService.getOne(new
QueryWrapper<GraphVertexDef>().eq("graph_id", graphId).eq("label", label));
                if (vertexDef != null) {
                    vertexLabel = vertexDef.getLabel();
                }
            }
            if (vertexLabel == null) {
                log.warn("顶点类型不存在，graphId={}, label={}", graphId, label);
                return false;
            }
            GraphVertex vertex = new GraphVertex();
            vertex.setUid(vertexId);
            vertex.setLabel(vertexLabel);
            graphDataOperations.deleteVertex(vertex);
            log.info("成功删除顶点，vertexId={}", vertexId);
            return true;
        } catch (Exception e) {
            log.error("删除顶点失败，graphId={}, vertexId={}", graphId, vertexId, e);
            return false;
        }
    }

    @Override
    public boolean deleteEdge(Long graphId, String edgeId, String label, Long connectionId, String graphCode) {
        if (graphId == null || edgeId == null) {
            return false;
        }
        try {
            GraphConf graphConf = GraphClientFactory.resolveGraphConf(graphId, connectionId, graphCode,
graphService, connectionService);

            GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
            GraphDataOperations graphDataOperations = graphClient.opsForGraphData();

            String edgeLabel = label;
            if (graphId > 0) {
```

```
                GraphEdgeDef edgeDef = edgeDefService.getOne(new QueryWrapper<GraphEdgeDef>().eq("graph_id",
graphId).eq("label", label));
                if (edgeDef != null) {
                    edgeLabel = edgeDef.getLabel();
                }
            }
            if (edgeLabel == null) {
                log.warn("边类型不存在，graphId={}, label={}", graphId, label);
                return false;
            }
            GraphEdge edge = new GraphEdge();
            edge.setUid(edgeId);
            edge.setLabel(edgeLabel);
            if (edgeId.contains("->") && edgeId.contains("@")) {
                int arrowIdx = edgeId.indexOf("->");
                int atIdx = edgeId.indexOf("@");
                if (atIdx > arrowIdx) {
                    edge.setStartUid(edgeId.substring(0, arrowIdx));
                    edge.setEndUid(edgeId.substring(arrowIdx + 2, atIdx));
                }
            } else if (edgeId.contains("->")) {
                int arrowIdx = edgeId.indexOf("->");
                edge.setStartUid(edgeId.substring(0, arrowIdx));
                edge.setEndUid(edgeId.substring(arrowIdx + 2));
            }

            if (edge.getStartUid() == null || edge.getEndUid() == null) {
                try {
                    String query = String.format("MATCH ()-[e:%s]->() WHERE e.uid == '%s' OR id(e) == '%s' RETURN
e", edgeLabel, edgeId, edgeId);
                    GraphData graphData = graphDataOperations.query(query);
                    if (graphData != null && graphData.getEdges() != null && !graphData.getEdges().isEmpty()) {
                        GraphEdge foundEdge = graphData.getEdges().get(0);
                        edge.setStartUid(foundEdge.getStartUid());
                        edge.setEndUid(foundEdge.getEndUid());
                    }
                } catch (Exception qe) {
                    log.warn("查询边数据失败，edgeId={}: {}", edgeId, qe.getMessage());
                }
            }

            if (edge.getStartUid() == null || edge.getEndUid() == null) {
                log.error("无法获取边的起点和终点信息，edgeId={}", edgeId);
                return false;
            }

            graphDataOperations.deleteEdge(edge);
            log.info("成功删除边，edgeId={}", edgeId);
            return true;
        } catch (Exception e) {
            log.error("删除边失败，graphId={}, edgeId={}", graphId, edgeId, e);
            return false;
        }
    }

    @Override
    public GraphSummary getGraphSummary(Long graphId, Long connectionId, String graphCode) {
        try {
            GraphConf graphConf = GraphClientFactory.resolveGraphConf(graphId, connectionId, graphCode,
graphService, connectionService);
```

```
            GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
            GraphDataOperations graphDataOperations = graphClient.opsForGraphData();
            GraphSummary summary = graphDataOperations.getSummary();
            summary.setGraphCode(graphConf.getGraphCode());
            return summary;
        } catch (Exception e) {
            log.error("获取图统计信息失败，graphId={}", graphId, e);
            throw new RuntimeException("获取图统计信息失败: " + e.getMessage(), e);
        }
    }

    private GraphDataOperations getGraphDataOperations(Long graphId, Long connectionId, String graphCode) {
        GraphConf graphConf = GraphClientFactory.resolveGraphConf(graphId, connectionId, graphCode, graphService,
connectionService);
        GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
        return graphClient.opsForGraphData();
    }

    private String buildGraphQuery(Long graphId, Long connectionId, String label, boolean isVertex, int skip, int size) {
        GraphInfo graphInfo = graphService.getById(graphId);
        GraphTypeEnum graphType = graphInfo != null
                ? graphInfo.getGraphType()
                : connectionService.getById(connectionId).getGraphTypeEnum();

        if (isVertex) {
            return switch (graphType) {
                case janus -> String.format("g.V().hasLabel(\"%s\").skip(%d).limit(%d)", label, skip, size);
                case nebula, neo4j -> String.format("MATCH (n:`%s`) RETURN n SKIP %d LIMIT %d", label, skip, size);
            };
        } else {
            return switch (graphType) {
                case janus -> String.format("g.E().hasLabel(\"%s\").skip(%d).limit(%d)", label, skip, size);
                case nebula, neo4j -> String.format("MATCH (a)-[r:`%s`]->(b) RETURN a, r, b SKIP %d LIMIT %d", label,
skip, size);
            };
        }
    }

    private String buildLabelQuery(Long graphId, Long connectionId, String label, int skip, int size) {
        return buildGraphQuery(graphId, connectionId, label, true, skip, size);
    }

    private String buildEdgeLabelQuery(Long graphId, Long connectionId, String label, int skip, int size) {
        return buildGraphQuery(graphId, connectionId, label, false, skip, size);
    }

    private Map<String, Object> handleProperties(Map<String, Object> data, List<GraphPropertyDef> propertyDefList) {
        Map<String, Object> properties = (Map<String, Object>) data.get("properties");
        if (propertyDefList != null) {
            for (GraphPropertyDef propDef : propertyDefList) {
                String code = propDef.getCode();
                if (properties.containsKey(code)) {
                    properties.put(code, DataTypeConverter.convertValue(properties.get(code), propDef.getType()));
                }
            }
        }
        properties.put(GraphConstants.UID, data.get(GraphConstants.UID));
        return properties;
    }
```

```
    public static Map<String, Object> extractProperties(Map<String, ?> raw, Set<String> excludeKeys) {
        Map<String, Object> result = new HashMap<>();
        if (raw == null) {
            return result;
        }
        raw.forEach((key, value) -> {
            if (excludeKeys == null || !excludeKeys.contains(key)) {
                result.put(key, value);
            }
        });
        return result;
    }
}
package com.chenpp.graph.core.model;
import lombok.Data;
import java.io.Serializable;
import java.util.Map;

@Data
public class GraphVertex implements Serializable {
    private String id;
    private String uid;
    private String label;
    private Map<String, Object> properties;
}

package com.chenpp.graph.core.model;
import lombok.Data;
import java.io.Serializable;
import java.util.Map;


@Data
public class GraphEdge implements Serializable {
    private static final long serialVersionUID = -7475762228551385446L;
    /**
     * 图库 id
     */
    private Serializable id;
    private String uid;
    private String label;
    private Map<String, Object> properties;
    private String startUid;
    private String startLabel;
    private String endUid;
    private String endLabel;
}

package com.chenpp.graph.nebula;
import com.chenpp.graph.core.GraphDataOperations;
import com.chenpp.graph.core.constant.GraphConstants;
import com.chenpp.graph.core.exception.ErrorCode;
import com.chenpp.graph.core.exception.GraphException;
import com.chenpp.graph.core.model.GraphData;
import com.chenpp.graph.core.model.GraphEdge;
import com.chenpp.graph.core.model.GraphSummary;
import com.chenpp.graph.core.model.GraphVertex;
import com.chenpp.graph.core.model.PathQuery;
import com.chenpp.graph.core.schema.DataType;
import com.chenpp.graph.core.schema.GraphEntity;
```

```
import com.chenpp.graph.core.schema.GraphProperty;
import com.chenpp.graph.core.schema.GraphRelation;
import com.chenpp.graph.nebula.schema.NebulaEdge;
import com.chenpp.graph.nebula.schema.NebulaProperty;
import com.chenpp.graph.nebula.schema.NebulaTag;
import com.chenpp.graph.nebula.util.NebulaUtil;
import com.vesoft.nebula.client.graph.SessionPool;
import com.vesoft.nebula.client.graph.data.PathWrapper;
import com.vesoft.nebula.client.graph.data.ResultSet;
import com.vesoft.nebula.client.graph.data.ValueWrapper;
import com.vesoft.nebula.client.graph.exception.IOErrorException;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.collections4.CollectionUtils;
import org.apache.commons.collections4.MapUtils;

import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;

@Slf4j
public class NebulaGraphDataOperations implements GraphDataOperations {
    private final NebulaConf nebulaConf;
    private final SessionPool sessionPool;

    public NebulaGraphDataOperations(NebulaConf nebulaConf) {
        this.nebulaConf = nebulaConf;
        this.sessionPool = NebulaClientFactory.getSessionPool(nebulaConf);
    }

    @Override
    public GraphVertex addVertex(GraphVertex vertex) throws GraphException {
        if (MapUtils.isEmpty(vertex.getProperties())) {
            throw new GraphException(ErrorCode.BAD_REQUEST, "Vertex properties cannot be empty");
        }
        try {
            Map<String, DataType> propertyTypes = getPropertyTypes(vertex.getLabel(), true);
            String nql = NebulaUtil.buildInsertVertex(vertex, propertyTypes);
            log.info("Execute NGQL: {}", nql);
            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                log.error("Failed to add vertex, errorCode: {}, errorMessage: {}",
                        resultSet.getErrorCode(), resultSet.getErrorMessage());
                throw new GraphException("Failed to add vertex, errorCode: " + resultSet.getErrorCode()
                        + ", errorMessage: " + resultSet.getErrorMessage());
            }
            return vertex;
        } catch (Exception e) {
            log.error("Failed to add vertex: {}", vertex, e);
            throw new GraphException("Failed to add vertex", e);
        }
    }

    @Override
```

```
    public GraphVertex updateVertex(GraphVertex vertex) throws GraphException {
        if (MapUtils.isEmpty(vertex.getProperties())) {
            throw new GraphException(ErrorCode.BAD_REQUEST, "Vertex properties cannot be empty");
        }
        try {
            String vid = vertex.getUid();
            Map<String, DataType> propertyTypes = getPropertyTypes(vertex.getLabel(), true);
            String setClause = vertex.getProperties().entrySet().stream()
                    .map(entry -> entry.getKey() + " = " + NebulaUtil.formatValue(entry.getValue(), propertyTypes != null ?
propertyTypes.get(entry.getKey()) : null))
                    .reduce((a, b) -> a + ", " + b)
                    .orElse("");
            String nql = NebulaUtil.buildUpdateVertex(vertex.getLabel(), vid, setClause);
            log.info("Execute NGQL: {}", nql);

            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                log.error("Failed to update vertex, errorCode: {}, errorMessage: {}",
                        resultSet.getErrorCode(), resultSet.getErrorMessage());
                throw new GraphException("Failed to update vertex, errorCode: " + resultSet.getErrorCode()
                        + ", errorMessage: " + resultSet.getErrorMessage());
            }
            return vertex;
        } catch (Exception e) {
            log.error("Failed to update vertex: {}", vertex, e);
            throw new GraphException("Failed to update vertex", e);
        }
    }

    @Override
    public void addVertices(Collection<GraphVertex> vertices) throws GraphException {
        if (CollectionUtils.isEmpty(vertices)) {
            log.info("Vertices collection is empty, skipping batch insert");
            return;
        }
        try {
            Map<String, List<GraphVertex>> verticesByLabel = vertices.stream()
                    .filter(vertex -> MapUtils.isNotEmpty(vertex.getProperties()))
                    .collect(Collectors.groupingBy(GraphVertex::getLabel));

            verticesByLabel.forEach((label, vertexList) -> {
                if (CollectionUtils.isEmpty(vertexList)) {
                    return;
                }
                String keys = String.join(",", vertexList.get(0).getProperties().keySet());
                Map<String, DataType> propertyTypes = getPropertyTypes(label, true);
                String valuesClause = vertexList.stream().map(vertex -> {
                    String propValues = NebulaUtil.buildPropertyValuesClause(vertex.getProperties(), propertyTypes);
                    return String.format("\"%s\":(%s)", vertex.getUid(), propValues);
                }).collect(Collectors.joining(","));
                String nql = NebulaUtil.buildInsertVertexBatch(label, keys, valuesClause);
                log.info("Execute batch insert NGQL: {}", nql);
                ResultSet resultSet;
                try {
                    resultSet = sessionPool.execute(nql);
                    if (!resultSet.isSucceeded()) {
                        log.error("Failed to batch insert vertices, errorCode: {}, errorMessage: {}",
                                resultSet.getErrorCode(), resultSet.getErrorMessage());
                        throw new GraphException("Failed to batch insert vertices, errorCode: " +
resultSet.getErrorCode()
```

```
                                + ", errorMessage: " + resultSet.getErrorMessage());
                    }
                } catch (Exception e) {
                    log.error("Failed to batch insert vertices", e);
                    throw new GraphException("Failed to batch insert vertices", e);
                }
            });
        } catch (Exception e) {
            log.error("Failed to batch insert vertices", e);
            throw new GraphException("Failed to batch insert vertices", e);
        }
    }

    @Override
    public boolean deleteVertex(GraphVertex vertex) throws GraphException {
        try {
            String nql = NebulaUtil.buildDeleteVertex(vertex.getUid());
            log.info("Execute NGQL: {}", nql);
            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                log.error("Failed to delete vertex, errorCode: {}, errorMessage: {}",
                        resultSet.getErrorCode(), resultSet.getErrorMessage());
                throw new GraphException("Failed to delete vertex, errorCode: " + resultSet.getErrorCode()
                        + ", errorMessage: " + resultSet.getErrorMessage());
            }
            return true;
        } catch (Exception e) {
            log.error("Failed to delete vertex: {}", vertex, e);
            throw new GraphException("Failed to delete vertex", e);
        }
    }

    @Override
    public GraphEdge addEdge(GraphEdge edge) throws GraphException {
        try {
            Map<String, DataType> propertyTypes = getPropertyTypes(edge.getLabel(), false);
            String keys = String.join(",", edge.getProperties().keySet());
            String propValues = NebulaUtil.buildPropertyValuesClause(edge.getProperties(), propertyTypes);
            String nql = NebulaUtil.buildInsertEdge(edge.getLabel(), keys, edge.getStartUid(), edge.getEndUid(),
propValues);
            log.info("Execute NGQL: {}", nql);
            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                log.error("Failed to add edge, errorCode: {}, errorMessage: {}",
                        resultSet.getErrorCode(), resultSet.getErrorMessage());
                throw new GraphException("Failed to add edge, errorCode: " + resultSet.getErrorCode()
                        + ", errorMessage: " + resultSet.getErrorMessage());
            }
            return edge;
        } catch (Exception e) {
            log.error("Failed to add edge: {}", edge, e);
            throw new GraphException("Failed to add edge", e);
        }
    }

    @Override
    public void addEdges(Collection<GraphEdge> edges) throws GraphException {
        if (CollectionUtils.isEmpty(edges)) {
            log.info("Edges collection is empty, skipping batch insert");
            return;
```

```
        }
        try {
            Map<String, List<GraphEdge>> edgesByLabel = edges.stream()
                    .collect(Collectors.groupingBy(GraphEdge::getLabel));

            for (Map.Entry<String, List<GraphEdge>> entry : edgesByLabel.entrySet()) {
                String label = entry.getKey();
                List<GraphEdge> edgeList = entry.getValue();
                if (edgeList.isEmpty()) {
                    continue;
                }
                GraphEdge firstEdge = edgeList.get(0);
                String propKeys = "";
                if (firstEdge.getProperties() != null && !firstEdge.getProperties().isEmpty()) {
                    propKeys = String.join(", ", firstEdge.getProperties().keySet());
                }
                Map<String, DataType> propertyTypes = getPropertyTypes(label, false);
                StringBuilder valuesBuilder = new StringBuilder();
                String valuesStr = edgeList.stream().map(edge -> {
                    String propValues = NebulaUtil.buildPropertyValuesClause(edge.getProperties(), propertyTypes);
                    return String.format("\"%s\" -> \"%s\":(%s)", edge.getStartUid(), edge.getEndUid(), propValues);
                }).collect(Collectors.joining(", "));
                valuesBuilder.append(valuesStr);
                String nql = NebulaUtil.buildInsertEdgeBatch(label, propKeys, valuesBuilder.toString());
                log.info("Execute batch insert NGQL: {}", nql);
                ResultSet resultSet = sessionPool.execute(nql);
                if (!resultSet.isSucceeded()) {
                    log.error("Failed to batch insert edges, errorCode: {}, errorMessage: {}",
                            resultSet.getErrorCode(), resultSet.getErrorMessage());
                    throw new GraphException("Failed to batch insert edges, errorCode: " + resultSet.getErrorCode()
                            + ", errorMessage: " + resultSet.getErrorMessage());
                }
            }
        } catch (Exception e) {
            log.error("Failed to batch insert edges", e);
            throw new GraphException("Failed to batch insert edges", e);
        }
    }

    @Override
    public GraphEdge updateEdge(GraphEdge edge) throws GraphException {
        try {
            Map<String, DataType> propertyTypes = getPropertyTypes(edge.getLabel(), false);

            String setClause = edge.getProperties().entrySet().stream()
                    .map(entry -> entry.getKey() + " = " + NebulaUtil.formatValue(entry.getValue(), propertyTypes != null ?
propertyTypes.get(entry.getKey()) : null))
                    .reduce((a, b) -> a + ", " + b)
                    .orElse("");
            String nql = NebulaUtil.buildUpdateEdge(edge.getLabel(), edge.getStartUid(), edge.getEndUid(), setClause);
            log.info("Execute NGQL: {}", nql);
            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                log.error("Failed to update edge, errorCode: {}, errorMessage: {}",
                        resultSet.getErrorCode(), resultSet.getErrorMessage());
                throw new GraphException("Failed to update edge, errorCode: " + resultSet.getErrorCode()
                        + ", errorMessage: " + resultSet.getErrorMessage());
            }

            return edge;
```

```
        } catch (Exception e) {
            log.error("Failed to update edge: {}", edge, e);
            throw new GraphException("Failed to update edge", e);
        }
    }

    @Override
    public boolean deleteEdge(GraphEdge edge) throws GraphException {
        try {
            String nql = NebulaUtil.buildDeleteEdge(edge.getLabel(), edge.getStartUid(), edge.getEndUid());
            log.info("Execute NGQL: {}", nql);
            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                log.error("Failed to delete edge, errorCode: {}, errorMessage: {}",
                        resultSet.getErrorCode(), resultSet.getErrorMessage());
                throw new GraphException("Failed to delete edge, errorCode: " + resultSet.getErrorCode()
                        + ", errorMessage: " + resultSet.getErrorMessage());
            }
            return true;
        } catch (Exception e) {
            log.error("Failed to delete edge: {}", edge, e);
            throw new GraphException("Failed to delete edge", e);
        }
    }

    @Override
    public GraphData query(String cypher) throws GraphException {
        try {
            log.info("Execute NGQL: {}", cypher);

            ResultSet resultSet = sessionPool.execute(cypher);
            if (!resultSet.isSucceeded()) {
                log.error("Failed to execute query, errorCode: {}, errorMessage: {}",
                        resultSet.getErrorCode(), resultSet.getErrorMessage());
                throw new GraphException("Failed to execute query, errorCode: " + resultSet.getErrorCode()
                        + ", errorMessage: " + resultSet.getErrorMessage());
            }

            Map<String, GraphVertex> vertexMap = new LinkedHashMap<>();
            Map<String, GraphEdge> edgeMap = new LinkedHashMap<>();

            for (int i = 0; i < resultSet.rowsSize(); i++) {
                ResultSet.Record record = resultSet.rowValues(i);
                for (ValueWrapper value : record.values()) {
                    if (value.isVertex()) {
                        GraphVertex vertex = NebulaUtil.parseVertex(value.asNode());
                        if (vertex.getUid() != null) {
                            vertexMap.put(vertex.getUid(), vertex);
                        }
                    }
                    if (value.isEdge()) {
                        GraphEdge edge = NebulaUtil.parseEdge(value.asRelationship());
                        edgeMap.put(edge.getUid(), edge);
                    }
                    if (value.isPath()) {
                        PathWrapper path = value.asPath();
                        for (com.vesoft.nebula.client.graph.data.Node node : path.getNodes()) {
                            GraphVertex vertex = NebulaUtil.parseVertex(node);
                            if (vertex.getUid() != null) {
                                vertexMap.put(vertex.getUid(), vertex);
```

```
                        }
                    }
                    for (com.vesoft.nebula.client.graph.data.Relationship rel : path.getRelationships()) {
                        GraphEdge edge = NebulaUtil.parseEdge(rel);
                        edgeMap.put(edge.getUid(), edge);
                    }
                }
            }
        }
        Set<String> vertexIds = vertexMap.values().stream().filter(v -> v.getProperties() == null)
                .map(GraphVertex::getUid).collect(Collectors.toSet());
        GraphData graphData = new GraphData();
        graphData.setVertices(new ArrayList<>(vertexMap.values()));
        graphData.setEdges(new ArrayList<>(edgeMap.values()));
        if (CollectionUtils.isNotEmpty(graphData.getEdges()) && CollectionUtils.isEmpty(graphData.getVertices())) {
            graphData.getEdges().forEach(e -> {
                vertexIds.add(e.getStartUid());
                vertexIds.add(e.getEndUid());
            });
        }
        if (CollectionUtils.isNotEmpty(vertexIds)){
            List<GraphVertex> vertices = this.getVerticesByIds(vertexIds);
            graphData.setVertices(vertices);
        }
        return graphData;
    } catch (Exception e) {
        log.error("Failed to execute query: {}", cypher, e);
        throw new GraphException(ErrorCode.GRAPH_QUERY_FAILED, e);
    }
}

public List<GraphVertex> getVerticesByIds(Collection<String> idList) throws GraphException {
    if (CollectionUtils.isEmpty(idList)) {
        return Collections.emptyList();
    }
    try {
        String idListStr = idList.stream().map(id -> "'" + id + "'").collect(Collectors.joining(", "));
        String ngql = NebulaUtil.buildMatchVertices(idListStr);
        log.info("Execute NGQL: {}", ngql);

        ResultSet resultSet = sessionPool.execute(ngql);
        if (!resultSet.isSucceeded()) {
            log.error("Failed to query vertices by IDs, errorCode: {}, errorMessage: {}",
                    resultSet.getErrorCode(), resultSet.getErrorMessage());
            throw new GraphException("Failed to query vertices by IDs, errorCode: " + resultSet.getErrorCode()
                    + ", errorMessage: " + resultSet.getErrorMessage());
        }
        List<GraphVertex> vertices = new ArrayList<>();
        for (int i = 0; i < resultSet.rowsSize(); i++) {
            ResultSet.Record record = resultSet.rowValues(i);
            for (ValueWrapper value : record.values()) {
                if (value.isVertex()) {
                    GraphVertex vertex = NebulaUtil.parseVertex(value.asNode());
                    vertices.add(vertex);
                }
            }
        }
        return vertices;
    } catch (IOErrorException e) {
        log.error("Failed to query vertices by IDs due to IO error", e);
```

```
            throw new GraphException("Failed to query vertices by IDs due to IO error", e);
        } catch (Exception e) {
            log.error("Failed to query vertices by IDs", e);
            throw new GraphException(ErrorCode.GRAPH_QUERY_FAILED, e);
        }
    }

    @Override
    public GraphData expand(String vertexId, int depth) throws GraphException {
        String ngql = NebulaUtil.buildExpandPath(vertexId, depth);
        return query(ngql);
    }

    @Override
    public GraphData expand(String label, String property, String value, int depth, int limit) throws GraphException {
        String ngql = String.format(
                "LOOKUP ON %s WHERE %s.%s == '%s' YIELD id(vertex) AS src"
                        + " | GO 1 TO %d STEPS FROM $-.src OVER * BIDIRECT YIELD $^ AS v1, $$ AS v2, edge AS e"
                        + " | LIMIT %d",
                label, label, property, value, depth, limit);
        log.debug("Executing expand by property: {}", ngql);
        return query(ngql);
    }

    @Override
    public GraphData findPath(String startVertexId, String endVertexId, int maxDepth) throws GraphException {
        String ngql = NebulaUtil.buildShortestPath(startVertexId, endVertexId, maxDepth);
        return query(ngql);
    }

    @Override
    public GraphData findPath(PathQuery pathQuery) throws GraphException {
        PathQuery.Condition start = pathQuery.getStartProperty();
        PathQuery.Condition end = pathQuery.getEndProperty();
        String startVid;
        String endVid;
        if (start.getProperty().equals(GraphConstants.UID)) {
            startVid = start.getValue();
        } else {
            GraphVertex startVertex = findVertex(start.getLabel(), start.getProperty(), start.getValue());
            if (startVertex == null) {
                log.warn("Start vertex not found: label={}, property={}, value={}",
                        start.getLabel(), start.getProperty(), start.getValue());
                throw new GraphException("Start vertex not found");
            }
            startVid = startVertex.getId();
        }
        if (GraphConstants.UID.equals(end.getProperty())) {
            endVid = end.getValue();
        } else {
            GraphVertex endVertex = findVertex(end.getLabel(), end.getProperty(), end.getValue());
            if (endVertex == null) {
                log.warn("End vertex not found: label={}, property={}, value={}",
                        end.getLabel(), end.getProperty(), end.getValue());
                throw new GraphException("End vertex not found");
            }
            endVid = endVertex.getId();
        }

        String pathNgql = String.format(
```

```
                "FIND ALL PATH FROM \"%s\" TO \"%s\" OVER * UPTO %d STEPS YIELD PATH AS p | LIMIT %d",
                startVid, endVid, pathQuery.getMaxDepth(), pathQuery.getLimit());
        log.info("Execute path query NGQL: {}", pathNgql);
        return query(pathNgql);
    }

    @Override
    public GraphSummary getSummary() throws GraphException {
        GraphSummary summary = new GraphSummary();
        try {
            String submitStatsJob = NebulaUtil.buildSubmitJobStats();
            ResultSet submitResult = sessionPool.execute(submitStatsJob);
            if (!submitResult.isSucceeded()) {
                log.error("Failed to submit STATS job, errorCode: {}, errorMessage: {}",
                        submitResult.getErrorCode(), submitResult.getErrorMessage());
                throw new GraphException("Failed to submit STATS job, errorCode: " + submitResult.getErrorCode()
                        + ", errorMessage: " + submitResult.getErrorMessage());
            }
            long jobId = submitResult.rowValues(0).get(0).asLong();
            String showJob = NebulaUtil.buildShowJob(jobId);
            ResultSet jobResult = null;
            int retryCount = 0;
            final int maxRetries = 30;
            while (retryCount < maxRetries) {
                Thread.sleep(1000);
                jobResult = sessionPool.execute(showJob);
                if (!jobResult.isSucceeded()) {
                    log.error("Failed to get job status, errorCode: {}, errorMessage: {}",
                            jobResult.getErrorCode(), jobResult.getErrorMessage());
                    throw new GraphException("Failed to get job status, errorCode: " + jobResult.getErrorCode()
                            + ", errorMessage: " + jobResult.getErrorMessage());
                }
                String status = jobResult.rowValues(1).get(2).asString();
                if ("FINISHED".equals(status)) {
                    break;
                }
                if ("FAILED".equals(status)) {
                    throw new GraphException("STATS job failed");
                }
                retryCount++;
            }

            if (retryCount >= maxRetries) {
                throw new GraphException("STATS job timeout");
            }

            String showStats = NebulaUtil.buildShowStats();
            ResultSet statsResult = sessionPool.execute(showStats);
            if (!statsResult.isSucceeded()) {
                log.error("Failed to show stats, errorCode: {}, errorMessage: {}",
                        statsResult.getErrorCode(), statsResult.getErrorMessage());
                throw new GraphException("Failed to show stats, errorCode: " + statsResult.getErrorCode()
                        + ", errorMessage: " + statsResult.getErrorMessage());
            }

            Map<String, Integer> vertexLabelCount = new HashMap<>();
            Map<String, Integer> edgeLabelCount = new HashMap<>();

            for (int i = 0; i < statsResult.rowsSize(); i++) {
                ResultSet.Record record = statsResult.rowValues(i);
```

```
                String type = record.get(0).asString();
                String name = record.get(1).asString();
                long count = record.get(2).asLong();

                switch (type) {
                    case "Tag" -> vertexLabelCount.put(name, (int) count);
                    case "Edge" -> edgeLabelCount.put(name, (int) count);
                    case "Space" -> {
                        if ("vertices".equals(name)) {
                            summary.setVertexCount((int) count);
                        } else if ("edges".equals(name)) {
                            summary.setEdgeCount((int) count);
                        }
                    }
                    default -> log.warn("Unknown stats type: {}", type);
                }
            }
            summary.setVertexLabelCount(vertexLabelCount);
            summary.setEdgeLabelCount(edgeLabelCount);
            summary.setGraphCode(nebulaConf.getGraphCode());
            return summary;
        } catch (Exception e) {
            log.error("Failed to get graph summary from Nebula", e);
            throw new GraphException("Failed to get graph summary from Nebula", e);
        }
    }

    @Override
    public long countVertices(String label) throws GraphException {
        String nql = NebulaUtil.buildCountVertex(label);
        try {
            ResultSet resultSet = sessionPool.execute(nql);
            if (resultSet.isSucceeded() && resultSet.rowsSize() > 0) {
                return resultSet.rowValues(0).get(0).asLong();
            }
        } catch (Exception e) {
            log.error("Failed to count vertices with label {} from Nebula", label, e);
        }
        return 0L;
    }

    @Override
    public long countEdges(String label) throws GraphException {
        String nql = NebulaUtil.buildCountEdge(label);
        try {
            ResultSet resultSet = sessionPool.execute(nql);
            if (resultSet.isSucceeded() && resultSet.rowsSize() > 0) {
                return resultSet.rowValues(0).get(0).asLong();
            }
        } catch (Exception e) {
            log.error("Failed to count edges with label {} from Nebula", label, e);
        }
        return 0L;
    }

    @Override
    public GraphVertex findVertex(String label, String property, String value) throws GraphException {
        try {
            String nql = NebulaUtil.buildFindVertexByProperty(label, property, value);
            log.info("Execute NGQL: {}", nql);
```

```
            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                return null;
            }
            for (int i = 0; i < resultSet.rowsSize(); i++) {
                ResultSet.Record record = resultSet.rowValues(i);
                for (ValueWrapper valueWrapper : record.values()) {
                    if (valueWrapper.isVertex()) {
                        return NebulaUtil.parseVertex(valueWrapper.asNode());
                    }
                }
            }
            return null;
        } catch (Exception e) {
            log.warn("Failed to find vertex by property: label={}, property={}, value={}, error={}",
                    label, property, value, e.getMessage());
            return null;
        }
    }

    private String edgeKey(GraphEdge edge) {
        if (edge.getUid() != null) {
            return edge.getUid();
        }
        return edge.getStartUid() + "->" + edge.getEndUid() + "#" + edge.getLabel();
    }

    private Map<String, DataType> getPropertyTypes(String label, boolean isVertex) {
        String nql = isVertex ? NebulaUtil.buildDescribeTag(label) : NebulaUtil.buildDescribeEdge(label);
        try {
            ResultSet resultSet = sessionPool.execute(nql);
            if (!resultSet.isSucceeded()) {
                log.warn("Failed to describe tag/edge {}, errorCode: {}, errorMessage: {}",
                        label, resultSet.getErrorCode(), resultSet.getErrorMessage());
                return null;
            }
            Map<String, DataType> propertyTypes = new HashMap<>();
            for (int i = 0; i < resultSet.rowsSize(); i++) {
                ResultSet.Record record = resultSet.rowValues(i);
                String fieldName = record.get(0).asString();
                String typeName = record.get(1).asString();
                DataType dataType = DataType.instanceOf(typeName);
                propertyTypes.put(fieldName, dataType);
            }
            return propertyTypes;
        } catch (Exception e) {
            log.warn("Failed to get property types for {}: {}", label, e.getMessage());
            return null;
        }
    }
}

package com.chenpp.graph.admin.controller;

import com.chenpp.graph.admin.model.GraphExpandRequest;
import com.chenpp.graph.admin.model.GraphPathRequest;
import com.chenpp.graph.admin.model.GraphQueryRequest;
import com.chenpp.graph.admin.model.Result;
import com.chenpp.graph.admin.service.GraphQueryService;
import com.chenpp.graph.core.model.GraphData;
```

```
import com.chenpp.graph.core.model.GraphSummary;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestParam;

/**
 * 图查询 API
 */
@Slf4j
@RestController
@RequestMapping("/api/graphs")
public class GraphQueryController {

    @Autowired
    private GraphQueryService graphQueryService;

    @PostMapping("/query")
    public Result<GraphData> query(
            @RequestBody(required = false) GraphQueryRequest request,
            @RequestParam(required = false) Long graphId,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode) {
        if (request == null) {
            request = new GraphQueryRequest();
        }
        if (request.getQuery() == null || request.getQuery().isEmpty()) {
            return Result.error("查询语句不能为空");
        }
        if (request.getGraphId() == null) request.setGraphId(graphId);
        if (request.getConnectionId() == null) request.setConnectionId(connectionId);
        if (request.getGraphCode() == null) request.setGraphCode(graphCode);

        GraphData graphData = graphQueryService.query(request);
        return Result.success(graphData);
    }

    @PostMapping("/expand")
    public Result<GraphData> expand(@RequestBody GraphExpandRequest request) {
        GraphData graphData = graphQueryService.expand(request);
        return Result.success(graphData);
    }

    @PostMapping("/path")
    public Result<GraphData> findPath(@RequestBody GraphPathRequest request) {
        GraphData graphData = graphQueryService.findPath(request);
        return Result.success(graphData);
    }

    @GetMapping("/summary")
    public Result<GraphSummary> getSummary(Long graphId, Long connectionId, String graphCode) {
        GraphSummary summary = graphQueryService.getSummary(graphId, connectionId, graphCode);
        return Result.success(summary);
    }
}
package com.chenpp.graph.admin.service.impl;
```

```
import com.chenpp.graph.admin.model.GraphExpandRequest;
import com.chenpp.graph.admin.model.GraphQueryRequest;
import com.chenpp.graph.admin.model.GraphPathRequest;
import com.chenpp.graph.admin.service.GraphConnectionService;
import com.chenpp.graph.admin.service.GraphQueryService;
import com.chenpp.graph.admin.service.GraphService;
import com.chenpp.graph.admin.util.GraphClientFactory;
import com.chenpp.graph.core.GraphDataOperations;
import com.chenpp.graph.core.constant.GraphConstants;
import com.chenpp.graph.core.model.GraphConf;
import com.chenpp.graph.core.model.GraphData;
import com.chenpp.graph.core.model.GraphSummary;
import com.chenpp.graph.core.model.PathQuery;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

/**
 * 图查询服务实现类
 */
@Slf4j
@Service
public class GraphQueryServiceImpl implements GraphQueryService {

    @Autowired
    private GraphService graphService;

    @Autowired
    private GraphConnectionService connectionService;

    @Override
    public GraphData query(GraphQueryRequest request) {
        GraphDataOperations graphDataOperations = resolveDataOps(request.getGraphId(), request.getConnectionId(),
request.getGraphCode());
        return graphDataOperations.query(request.getQuery());
    }

    @Override
    public GraphData expand(GraphExpandRequest request) {
        Long graphId = request.getGraphId();
        String vertexId = request.getVertexId();
        Integer depth = request.getDepth();
        String label = request.getLabel();
        String property = request.getProperty();
        Long connectionId = request.getConnectionId();
        String graphCode = request.getGraphCode();
        if (StringUtils.isBlank(vertexId)) {
            throw new IllegalArgumentException("查询值不能为空");
        }
        GraphDataOperations graphDataOperations = resolveDataOps(graphId, connectionId, graphCode);

        if (StringUtils.isNoneBlank(label, property) && !GraphConstants.UID.equals(property)) {
            return graphDataOperations.expand(label, property, vertexId, depth, 100);
        }
        return graphDataOperations.expand(vertexId, depth);
    }

    @Override
```

```
    public GraphData findPath(GraphPathRequest request) {
        Integer maxDepth = request.getMaxDepth();
        if (maxDepth == null) {
            maxDepth = 5;
        }
        boolean hasStartPropCondition = StringUtils.isNoneBlank(request.getStartLabel(), request.getStartProp(),
request.getStartValue());
        boolean hasEndPropCondition = StringUtils.isNoneBlank(request.getEndLabel(), request.getEndProp(),
request.getEndValue());

        GraphDataOperations ops = resolveDataOps(request.getGraphId(), request.getConnectionId(),
request.getGraphCode());

        if (hasStartPropCondition && hasEndPropCondition) {
            PathQuery pq = new PathQuery();
            pq.setStartProperty(new PathQuery.Condition(request.getStartLabel(), request.getStartProp(),
request.getStartValue()));
            pq.setEndProperty(new PathQuery.Condition(request.getEndLabel(), request.getEndProp(),
request.getEndValue()));
            pq.setMaxDepth(maxDepth);
            pq.setLimit(1000);
            return ops.findPath(pq);
        }
        if (StringUtils.isNotBlank(request.getStartVertexId()) && StringUtils.isNotBlank(request.getEndVertexId())) {
            return ops.findPath(request.getStartVertexId(), request.getEndVertexId(), maxDepth);
        }
        throw new IllegalArgumentException("请提供起点和终点的查询条件（属性条件或顶点 ID）");
    }

    @Override
    public GraphSummary getSummary(Long graphId, Long connectionId, String graphCode) {
        return resolveDataOps(graphId, connectionId, graphCode).getSummary();
    }

    private GraphDataOperations resolveDataOps(Long graphId, Long connectionId, String graphCode) {
        GraphConf graphConf = GraphClientFactory.resolveGraphConf(
                graphId, connectionId, graphCode, graphService, connectionService);
        return GraphClientFactory.createGraphClient(graphConf).opsForGraphData();
    }

}

package com.chenpp.graph.core.model;
import lombok.Data;
import java.util.Map;

/**
 * 图数据统计
 */
@Data
public class GraphSummary {
    private String graphCode;
    private String graphName;
    private int vertexCount;
    private int edgeCount;
    private Map<String, Integer> vertexLabelCount;
    private Map<String, Integer> edgeLabelCount;
}
```